Technical approach

Apex Ridge Capital | Prototype handoff | Updated October 12, 2026

I built a local application that turns SEC filings into a reviewable fund comparison. It automates finding and interpreting disclosures; the PM still decides whether the comparison is ready, and compliance can follow each accepted number back to its source.

The scope I used

The comparison is for Q4 2024, using filings published by February 28, 2025. Each number keeps its actual reporting date, even when it is older than the target quarter. Competitor NAV history covers eight quarter-end observations; other metrics use the latest supported values. The project owner authorized the supplied Apex Ridge CSV for this prototype: Q4 2024 metrics and Q1-Q4 2024 NAV history are included; 2025 rows are excluded.

1. Find the filings

The Python service connects to SEC EDGAR, lists filings by fund identifier (CIK), and downloads eligible reports, supplements and exhibits. It uses a declared contact, rate limiting and bounded retries. The fund browser lets a user search for another fund and specify its structure and share class before adding it.

2. Parse once and save the source

HTML tables, narrative text, footnotes and PDF text become a reusable passage index. I save the original documents, their SHA-256 fingerprints and source locations alongside it. PDF citations use page numbers; HTML citations use table or text locations. N-PORT XML has dedicated field mappings and balance checks. Scanned PDF pages can use local OCR, but their numbers require visual confirmation.

3. Retrieve evidence for one question

A metric definition identifies relevant passages from the saved index. The retrieval step preserves table headings and nearby footnotes. For effective fees, it removes repeated context and excludes unrelated legal waiver language. Custom metrics use a user-written definition, units, scope and search phrases; they currently support reported values, not arbitrary formulas.

4. Let the LLM interpret; let code check

GPT-6.1 Sol receives the evidence and comparison rules for one result. It selects a value or explains why it cannot, with numbered source references. Code then checks those references and the proposed dates, units and calculations. The next page describes that division of work.

5. Save the decision, then produce the report

The board Word copy shows comparisons, observation dates and PM-authored commentary. The separate compliance copy and dashboard retain sources, definitions and rationales. Both read saved decisions. CSV, PDF and JSON exports are also available. Opening a report or downloading Word does not call the model or repeat extraction.

Selecting and checking values

The model makes the interpretation visible. It does not approve the board package.

What the LLM is responsible for

Sol interprets the disclosures, compares competing figures and explains its choice. It uses OpenAI's Responses API with structured JSON output and store=false. Only public SEC evidence enters requests. Each affected result gets one initial review; there is no routine second LLM verifier. A failed code check permits one correction attempt, with both attempts retained.

What the code verifies

The model selects numbered passages; code restores their exact saved text. It checks source fingerprints, filing dates, values, periods, units and share classes. Calculations must use implemented formulas with cited inputs. Reconciliations check N-PORT assets minus liabilities against net assets, compound annual return chains, and distinguish rounding differences from conflicting figures. These checks do not prove every interpretation is right.

How confidence appears in the output

I use evidence status and a written rationale, not high/medium/low grades. A supported value has source links. A blank distinguishes no retrieved evidence, unfinished or failed review, and a reviewed but unestablished value. A failed search is not labeled "not disclosed."

A concrete example: Golub management fees

Disclosure	Figure	How it is treated
Contractual management fee	1.00%	Kept as the primary contractual rate, on average adjusted gross assets.
Annual management-fee expense ratio	1.56%	Separate effective-fee observation for September 30, 2024, on period-end net assets attributable to common stock.
Older expense table in a prospectus	2.75%	Not selected: its footnote refers to March 2022 and excludes a one-time waiver.

Sol selected 1.56% with the older date and different denominator. This single-case trial does not establish overall financial accuracy.

Sources: Golub's 2024 annual report, Fees and Expenses, footnote 4, and November 2024 prospectus, fee-table footnote 4. Full passages and the trial result are retained in the audit.

When two figures really conflict

Different periods or fee bases may explain a difference. Temporary waivers retain their stated expiration. For the same measure, period and class, the model needs an explicit correction or explanation; a newer filing alone is not enough. Unresolved conflicts show both figures and citations for PM review, with no accepted value.

Storage and production design

Current file layout, the core records, and a path to a shared service.

Current storage and schema

The current implementation uses files, not a database. Original filings live in .cache/sec/, named by content hash, with JSON metadata mapping URLs to files. Separate folders hold passages and model decisions. Each report's audit.json captures its configuration, results, history, evidence and errors. Its logical schema is:

Record	Key fields and relationships
Filing and document	Fund CIK, accession, form, filing/reporting dates, URL and content hash. A filing has one or more documents.
Passage	ID tied to document hash and parser version; text, table rows, headings and source locator.
Decision	Fund, metric, period, value, unit, basis, class, evidence IDs, rationale, validation status and model provenance.

Cache keys include evidence, comparison rules and model. Metric edits reuse parsed documents and review only affected results, with three concurrent workers by default. Exports read saved decisions.

Proposed production storage

I would use PostgreSQL for funds, filings, versioned metric definitions, decisions and approvals. Citation records would link decisions to passages and documents; versioned object storage would hold original filings and exports. Decisions would be appended, preserving approved history, and reports would reference specific decision versions. This production design is proposed, not implemented.

Validation and remaining work

What has been checked

Automated regression tests pass across source validation, dates, classes, tables, calculations, conflicts and reuse. Checks against real filings caught a hurdle mistaken for an incentive fee. The Sol fee trial took 18.9 seconds: one model call, no reparsing and 104 other results unchanged. This measures a cached update, not cold ingestion.

Before production

Confirm the internal source with the client for production, complete a full Sol run and compare a quarterly package with the analyst's work. Build a human-checked evaluation of present, absent, conflicting and unreadable disclosures; measure correct values, valid citations and missed figures. No absence-recall benchmark has yet been completed. Unfamiliar layouts and OCR still require review; arbitrary custom calculations are not supported. Add authenticated approvals, backups and alerts for failed retrieval or model calls before routine use.

Implementation: src/apex_benchmark/. Measured runs and validation records: docs/.